

CSE-2421 - Computer Algorithm

Midterm Syllabus

Segment 1: Algorithm Analysis, Divide and Conquer

Algorithms and their analysis: definition and properties of algorithms; application areas of algorithms; correctness proof of algorithms (using loop invariant of insertion sort and others); complexity analysis of iterative algorithms; growth of functions and asymptotic notations.

Divide and conquer: mergesort; finding running time of recursive algorithm using substitution method, recursion tree method, and master method; quicksort, analysis of quicksort, randomized quicksort; lower bound of sorting.

Segment 2: Greedy Algorithms, String Algorithms

Greedy algorithms: activity selection problem; fractional knapsack problem; elements of greedy strategy; Huffman codes; maximum subarray problem: brute-force, divide-and-conquer and greedy (Kadane's algorithm).

String-matching algorithms: naive string-matching algorithm; Rabin-Karp algorithm.

Segment 3: Dynamic Programming (Foundation)

Dynamic programming (foundation): Fibonacci numbers: introducing memoization and tabulation; climbing stairs: basic 1D state transition; rod cutting problem: maximize profit by cutting a rod into different lengths; coin change; 0/1 Knapsack; steps to develop a dynamic programming algorithm, elements of dynamic programming.

Practice Questions

Segment 1: Algorithm Analysis, Divide and Conquer

Part 1: Algorithm Fundamentals

1. **Definitions:** Define an algorithm. What are the key properties of a good algorithm?
2. **Comparisons:** Differentiate between the following concepts:
 - a. Time complexity vs. Space complexity
 - b. Worst-case vs. Average-case analysis

Part 2: Asymptotic Analysis & Growth of Functions

1. **Function Bounding:** Given $f(n) = 2n + 80$ and $g(n) = n \log n$, is $f(n)$ an upper bound or a lower bound of $g(n)$? Justify your answer formally.
2. **Big-O Notation:** Given $f(n) = 4n + 15$ and $g(n) = 3n^2 + 2$, is $g(n) \in O(f(n))$? Justify your answer mathematically.
3. **Growth Rates:** Arrange the following functions in increasing order of growth and justify your arrangement:

$$n^2, \quad 2^n, \quad n!, \quad n^3$$

Part 3: Recurrence Relations

1. Master Theorem

Find the asymptotic bound of the following recurrence relations using the Master Theorem:

1. $T(n) = 7T(n/2) + 3n^2$
2. $T(n) = 3T(n/2) + 2n$
3. $T(n) = 6T(n/2) + n^3$
4. $T(n) = 5T(n/3) + n^2$
5. $T(n) = 8T(n/2) + cn^3$

2. Recursion Tree Method

Find the asymptotic bound of the following recurrence relations using the Recursion Tree Method:

1. $T(n) = 2T(n/3) + n^2$
2. $T(n) = 2T(n/2) + n$
3. $T(n) = T(n/4) + T(n/2) + n^2$
4. $T(n) = T(n/3) + T(2n/3) + cn$

3. Substitution Method / General Recurrences

Find the asymptotic bound of the following recurrence relations using the Substitution Method (or Master Theorem where applicable):

1. $T(n) = T(n/2) + c$
2. $T(n) = 2T(n/2) + c$
3. $T(n) = 2T(n/2) + n$
4. $T(n) = nT(n-1)$
5. $T(n) = T(n-1) + \log n$

Part 4: Time Complexity Analysis

Analyze and find the time complexity of the following code snippets. Show your reasoning.

```
// Snippet A
double fn(double n) {
    if (n <= 1) return 0;
    return n * fn(n / 2);
}

// Snippet B
int fact(int n) {
    if (n == 1) return 1;
    return n * fact(n - 1);
}

// Snippet C
void P(int n) {
    int sum = 0;
    for (int i = 1; i <= log(n); i++) {
        for (int j = 1; j * j <= i; j++) {
            sum += j + i;
        }
    }
}

// Snippet D
void printPairs(int n) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            printf("%d, %d\n", i, j);
        }
    }
}

// Snippet E
void complexLoop(int n) {
    for (int i = 1; i < n; i *= 2) {
        int k = 0;
        while (k <= sqrt(n)) {
            k++;
        }
    }
}
```

Part 5: Correctness Analysis

1. Consider the standard Insertion Sort algorithm. State a valid loop invariant for the outer loop and use the three properties to prove that the algorithm correctly sorts an array $A[1 \dots n]$.

Part 6: Sorting Algorithms

1. **Algorithm Comparison:** Compare Quick Sort and Merge Sort. In which specific scenarios does one perform better than the other?
2. **Quick Sort Recurrences:** What will be the recurrence relation of Quick Sort for the following scenarios?
 - a. The input array is sorted in reverse order.
 - b. Each split partitions the array in a 90: 10 ratio.
3. **Partitioning Process:** Show the step-by-step execution of the *first partition* of the array $A = \{20, 2, 9, 7, 12, 15, 1, 5, 8\}$ using:
 - a. Pivot = 8 (Last element)
 - b. Pivot = 20 (First element)
4. **Limitations:** What is the primary limitation of Quick Sort, and what strategies can be used to overcome it?
5. **Lower Bounds:** What is the lower bound of comparison-based sorting? Prove the lower bound mathematically.

Segment 2: Greedy Algorithms, String Algorithms

Theoretical Questions

1. Given a set of activities with start and finish times, explain why selecting activities based on earliest finish time always leads to an optimal solution. Provide a counterexample for choosing earliest start time.
2. Explain why sorting items based on value/weight ratio guarantees optimality in the fractional knapsack problem.
3. Compare fractional knapsack with 0/1 knapsack. Why does greedy fail in 0/1 knapsack but succeed in fractional?
4. Provide a real-world problem where greedy strategy fails. Explain why greedy decisions do not lead to a global optimum.
5. Explain how Huffman coding minimizes the total encoding cost. Why is it considered a greedy algorithm?
6. What is the role of a priority queue in Huffman coding? Analyze its impact on time complexity.
7. Explain the divide-and-conquer approach for the maximum subarray problem. Why do we consider crossing subarrays?
8. Prove that Kadane's algorithm works correctly using the concept of optimal substructure.
9. Provide an example where naive string matching performs poorly and explain why.

10. Consider a text compression system using Huffman coding; analyze why merging the two least frequent symbols at each step leads to minimum weighted path length in the resulting tree, and explain how this reduces the total encoding cost.
11. Suppose a greedy algorithm makes locally optimal choices at each step; analyze under what conditions these local choices lead to a globally optimal solution, and provide reasoning using the concepts of optimal substructure and greedy choice property.
12. Consider an array where all elements are negative; analyze how Kadane's algorithm behaves in this case and explain how the algorithm ensures correctness.
13. Provide examples where the naive string-matching algorithm performs best and worst, with explanations. Also, mention the time complexity for each case.
14. In which type of input scenario does Rabin-Karp perform no better than the naive string-matching algorithm? Explain why.

Mathematical Questions

1. Given the following activities:

Activity	A1	A2	A3	A4	A5	A6	A7	A8
Start	1	3	0	5	3	5	6	8
Finish	4	5	6	7	9	9	10	11

Select the maximum number of non-overlapping activities using greedy method.

2. Solve using fractional knapsack and find maximum value. Knapsack capacity = 50.

Item	Weight	Value
I1	10	60
I2	20	100
I3	30	120

3. For the given set of items and the knapsack capacity of 15 kg, find the subset of the items to be added in the knapsack such that the profit is maximum.

Objects	1	2	3	4	5	6	7
Profit	10	5	15	7	6	18	3
Weight	2	3	5	7	1	4	1

4. Given array: [-2, 1, -3, 4, -1, 2, 1, -5, 4]. Find maximum subarray sum using:

- (a) Kadane's algorithm (Greedy)
- (b) Divide & conquer, show the subarray.

5. Find the maximum subarray sum of array = **[4, -5, 3, -2, 1, 5, -6, 3]**, using Kadane's Algorithm.
6. Consider the famous quote by Albert Einstein: **"Learn from yesterday."** Construct the Huffman tree for this sentence by first determining the frequency of each character (including spaces), then assign binary codes and encode the full sentence. Finally, calculate the total number of bits required using Huffman coding and compare it with fixed-length encoding (8 bits per character) to determine the compression ratio.
7. Consider the text **"ABABDABACDABABCABAB"** and the pattern **"ABABCABAB."** Using the naive string-matching algorithm, show all possible alignments of the pattern with the text, determine at which positions the pattern matches successfully, and calculate the total number of character comparisons performed during the process.
8. Consider the string **"UTTXYYYYXGGGUXXXTTYYY"**. Apply both fixed-length and variable-length Huffman coding to encode the string. Compare the total number of bits required in each case and show that variable-length coding is more efficient.
9. Given the text **T = "CABBCAACBAA"** and the pattern **P = "BCAA"**, apply the Rabin-Karp algorithm using a rolling hash function with modulus **q = 3**. Compute the hash values for the pattern and each window of the text, show all rolling hash updates, and indicate where exact match and spurious hits occur (if any).

Segment 3: Dynamic Programming (Foundation)

1.	You can climb either 1 or 2 steps at a time. Let $W(n)$ denote the number of distinct ways to reach step n, starting from step 0. (a) Formulate the recurrence relation and base cases for $W(n)$. (b) Compute $W(5)$, showing the intermediate steps (table or recursion tree). (c) State the time and space complexity														
2.	Greedy/DP/both works for 0/1 knapsack. Prove or disprove.														
3.	There are N items numbered 1, 2, ..., N. For each i ($1 \leq i \leq N$), Item i has a weight of w_i and a value of v_i. You have to decide which items to choose from the N items and carry them in a knapsack. The capacity of the knapsack is W. Let $W = 15$, $N = 6$. <table><tr><td>w_i</td><td>6</td><td>5</td><td>6</td><td>6</td><td>3</td><td>7</td></tr><tr><td>v_i</td><td>5</td><td>6</td><td>4</td><td>6</td><td>5</td><td>2</td></tr></table>	w_i	6	5	6	6	3	7	v_i	5	6	4	6	5	2
w_i	6	5	6	6	3	7									
v_i	5	6	4	6	5	2									

4.	Consider a money system consisting of 3 coins. The coins are {2,3,5} and the desired sum is 5, there are 3 ways to get desired money. 1. 5 2. 2 + 3 3. 3 + 2 Now Calculate the number of distinct ways you can produce a money sum 9 using the available coins.								
5.	For the coin change problem with arbitrary denominations, the greedy strategy of always choosing the largest coin may not yield the optimal solution. Provide a specific example (coin denominations and target amount) where the greedy approach fails.								
6.	Given a problem, what key elements help you decide whether it can be solved using dynamic programming? Explain briefly.								
7.	Given a Staircase of size n. You can take 1, 3 or 4 steps at a time. You have to _ _ _ _ Find the minimum number of steps to reach n. Among greedy or dp solutions, how will you approach the problem. Mention the greedy choice in case you chose greedy, or dp states (base case, recurrence relation) in case you chose dp.								
8.	Given an $n \times n$ grid, you can move only right or down. Find the minimum cost path from the top-left to bottom-right. (a) Define the DP state. (b) Write the recurrence relation. (c) State the base cases.								
9.	You are given a rod of length 3. You may cut the rod at any integer position (cuts are free). The prices for each possible piece length are given below: <table><tr><td>Piece Length</td><td>1</td><td>2</td><td>3</td></tr><tr><td>Price</td><td>2</td><td>5</td><td>7</td></tr></table> The total revenue is the sum of the prices of all pieces obtained after cutting. Each piece must have a positive integer length, and the sum of all piece lengths must equal 5. Questions (a) Use the bottom-up dynamic programming approach to find the maximum revenue $R(5)$. Show the 1D table as it fills up from $R(0)$ to $R(5)$. (b) Describe one best cutting plan (how many pieces and length of each piece). (c) Draw the full recursion tree for a naive top-down approach for $n = 4$ using the prices above. (d) Highlight the overlapping subproblems in your tree and explain why memoization reduces the time complexity.	Piece Length	1	2	3	Price	2	5	7
Piece Length	1	2	3						
Price	2	5	7						

10.	A rod of length n has a price array $p[1..n]$, where $p[i]$ represents the price of a piece of length i . Formulate a dynamic programming recurrence to determine the maximum obtainable revenue $R(n)$. Also write the pseudo code.
11.	Discuss the differences between Top-down and Bottom-up Dynamic Programming approach. Compare Time and Space complexity? How do you determine the ideal approach among the two? State your reasoning.
12.	Define Memoization in the context of Dynamic Programming. Explain its underlying principle and analyze why memoization is essential for improving computational efficiency.